Data Structures: Design & Analysis

Akmaral Abilkhaiyr | SE-2518 | Design and Analysis of Algorithms

DynamicArray and MinHeap use primitive int arrays, initial capacity 8 and 2x growth. MyLinkedList is singly linked with head/tail pointers. Both sequences share IntSequence; no production collection or boxed element storage is used. Capacity is not reduced after removals.

n = current size; C = capacity. Total storage: array/heap Θ(C), list Θ(n). For insertion-only growth C = Θ(n+1); after deletion it reflects the historical maximum. Average indexed bounds assume uniform indices; search assumes uniform hit position or constant miss probability. Heap averages assume random distinct keys.

Structure / operation	Best	Average	Worst	Aux. worst	Justification
Array add(x)	Θ(1)	Θ(1) amortized	Θ(n)	Θ(n)	Geometric copies total Θ(m) over m appends; one resize copies n.
Array add(i,x)	Θ(1)	Θ(n)	Θ(n)	Θ(n)	Shift n-i entries; growth can also copy n.
Array remove(i)	Θ(1)	Θ(n)	Θ(n)	Θ(1)	Shift n-i-1 entries; no shrinking.
Array get(i)	Θ(1)	Θ(1)	Θ(1)	Θ(1)	One indexed read.
Array contains(x)	Θ(1)	Θ(n)	Θ(n)	Θ(1)	Stop at first match or scan all n values.
List add(x)	Θ(1)	Θ(1)	Θ(1)	Θ(1)	Tail pointer avoids traversal; allocate one node.
List add(i,x)	Θ(1)	Θ(n)	Θ(n)	Θ(1)	Head/tail are special cases; middle visits predecessor.
List remove(i)	Θ(1)	Θ(n)	Θ(n)	Θ(1)	Head is direct; other positions need predecessor traversal.
List get(i)	Θ(1)	Θ(n)	Θ(n)	Θ(1)	Exactly i next-link traversals, Θ(i+1) runtime.
List contains(x)	Θ(1)	Θ(n)	Θ(n)	Θ(1)	Compare sequential nodes until match or null.
Heap insert(x)	Θ(1)	Θ(1) expected amortized*	Θ(n)	Θ(n)	Sift up is O(log n); a resizing call copies n.
Heap peekMin()	Θ(1)	Θ(1)	Θ(1)	Θ(1)	Root read without mutation.
Heap extractMin()	Θ(1)	Θ(log n)	Θ(log n)	Θ(1)	Replacement may stop at root or descend the height.
Heap buildHeap(a)	Θ(n)	Θ(n)	Θ(n)	Θ(n)	Defensive copy plus bottom-up sifts.

Θ gives matching O and Ω bounds. *Random-order heap insertion has expected amortized Θ(1); without that distribution the amortized bound is O(log n), Ω(1). A full-capacity call is Θ(n) because of copying; without growth its worst case is Θ(log n). Auxiliary space includes new arrays/nodes: non-growing insertion uses Θ(1). Invalid operations, size() and metrics() take Θ(1); diagnostic toArray() takes Θ(n) time and space.

Measurement protocol

Seed 42; n = 100, 1,000, 10,000, 100,000; 10 discarded warm-ups plus 5 measured fresh instances per case. Median time is saved with deterministic counters; all 260 raw runs are retained. Shared data and queries ensure fairness. W1: 10,000 gets. W2: 500 guaranteed hits and 500 misses. W3: 1,000 inserts then 1,000 removals at head or fixed original n/2. W4: n inserts then n extracts, sorted validation outside timing. Setup is excluded for W1-W3; measured growth is included. OpenJDK 25.0.1, Windows 11, 12 logical processors; one JVM process, instrumented timings.

Correctness: Two Loop Invariants

Akmaral Abilkhaiyr | SE-2518 | Design and Analysis of Algorithms

A. DynamicArray.contains

Invariant. Before iteration i, 0 ≤ i ≤ size, and every position j in [0, i) contains a value different from the searched value; the array and size are unchanged.

Initialization. At i=0 the examined prefix is empty, so the statement is vacuously true.

Maintenance. Read data[i]. If it equals the value, returning true is justified by that witness. Otherwise all positions in [0, i+1) differ from the value; incrementing i preserves the invariant.

Termination. The variant size-i decreases by one on each continuing iteration. If the loop exhausts at i=size, the invariant covers every stored position, so returning false is correct. For an empty array, the same conclusion holds immediately.

Conclusion. The method returns true exactly when a stored element equals the query and never changes the sequence.

B. MinHeap.bubbleDown

Precondition. For the invocation's root r, its left and right subtrees are heaps; only r's outgoing edges may be unordered. Extraction establishes this after moving the last value to the root. Bottom-up construction establishes it because children are processed before parents.

Invariant. Within the subtree rooted at r, all parent-child edges except those from the current index are ordered, and its multiset is unchanged. The cached value equals the value at the current index. If the current index is below r, its parent is no greater than every value in its subtree.

Initialization. The current index is r. Both child subtrees are heaps by the precondition; the ancestor condition is vacuous, and reading the root changes no value.

Maintenance. Select the smaller child, including the one-child case. If value is greater, swapping places the smallest of the parent and both child roots at the old index. The previously valid subheaps show that this promoted value is no greater than any value below it. The ancestor condition prevents a violation above the swap. Only the demoted value's outgoing edges can now violate order, so descending to that child preserves every part of the invariant.

Termination. Each swap moves one level downward in a finite tree. At a leaf there is no outgoing edge to repair. If value <= childValue, it is no greater than either child and the invariant already guarantees all other edges. Thus the complete subtree rooted at r is a heap.

Conclusion. Bubble-down restores heap order while preserving the multiset. Hence extraction removes precisely the old minimum, and Floyd's decreasing-parent loop builds a correct heap.

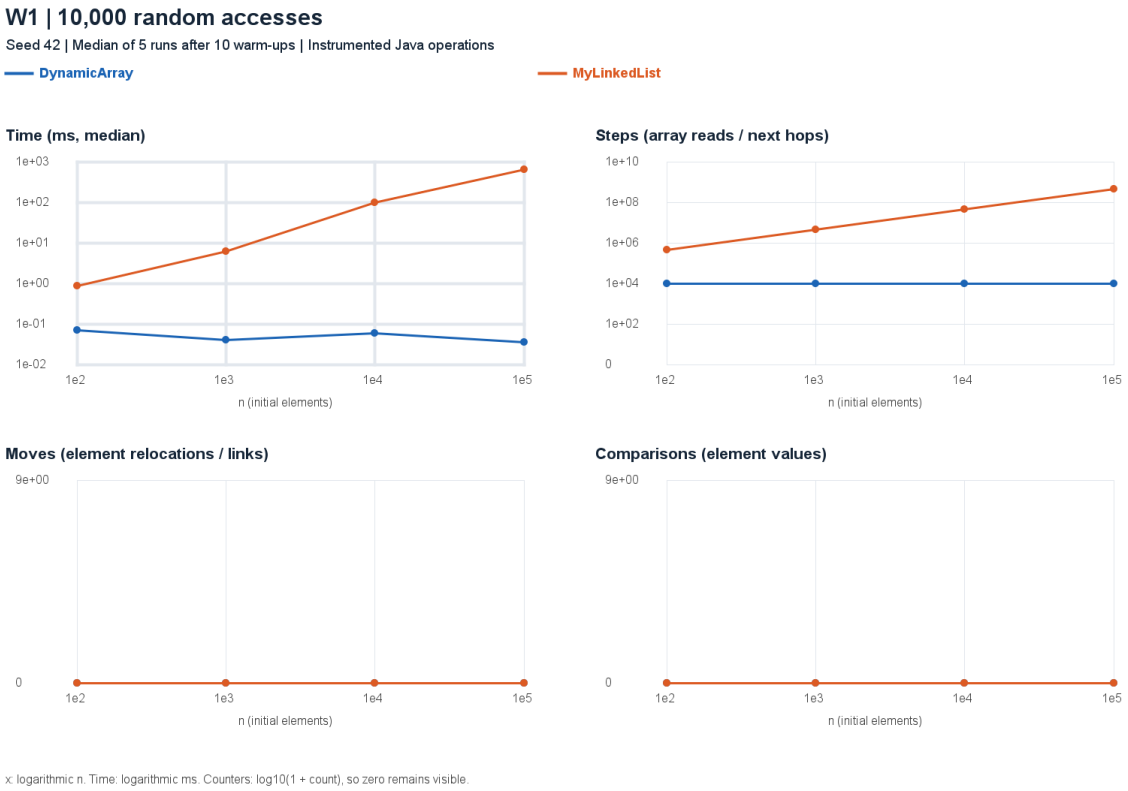
Physical-operation counters and tests

Steps = array-cell reads or next-link traversals (including null). Moves = existing-value relocations, growth copies, two heap-swap writes, or explicit head/tail/next updates. New array input writes are not moves; Floyd input copies are. Comparisons = element-value comparisons. Local assignments, bounds tests, reference comparisons, node value reads and allocations are excluded. Thus list get(0) has zero steps, while head W3 still has 3,000 link updates. Diagnostic toArray() does not alter counters.

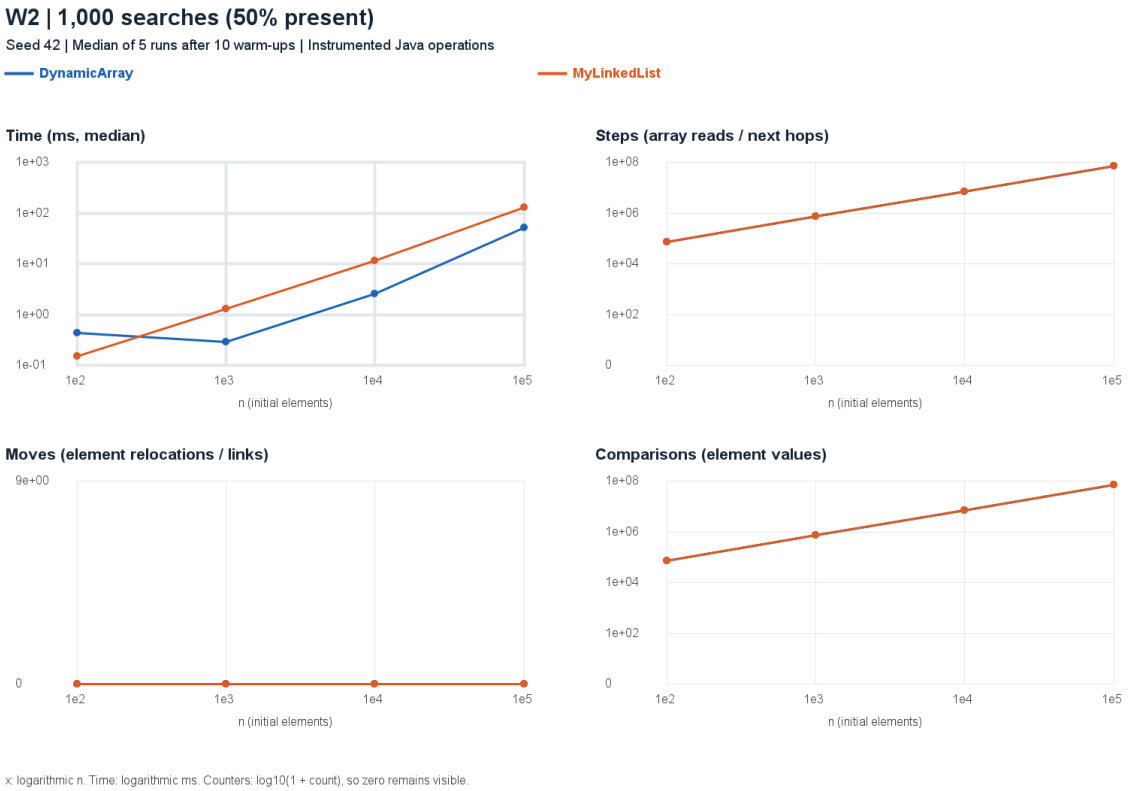
All 18 JUnit 5 tests pass: random reference comparisons, empty/singleton/duplicate/extreme values, invalid indices, selected exact counters, heap property after every tested mutation, sorted extraction and defensive Floyd construction.

Results: Access & Search

Same input for both structures | Time is median of 5 measured runs | Operation counts are deterministic



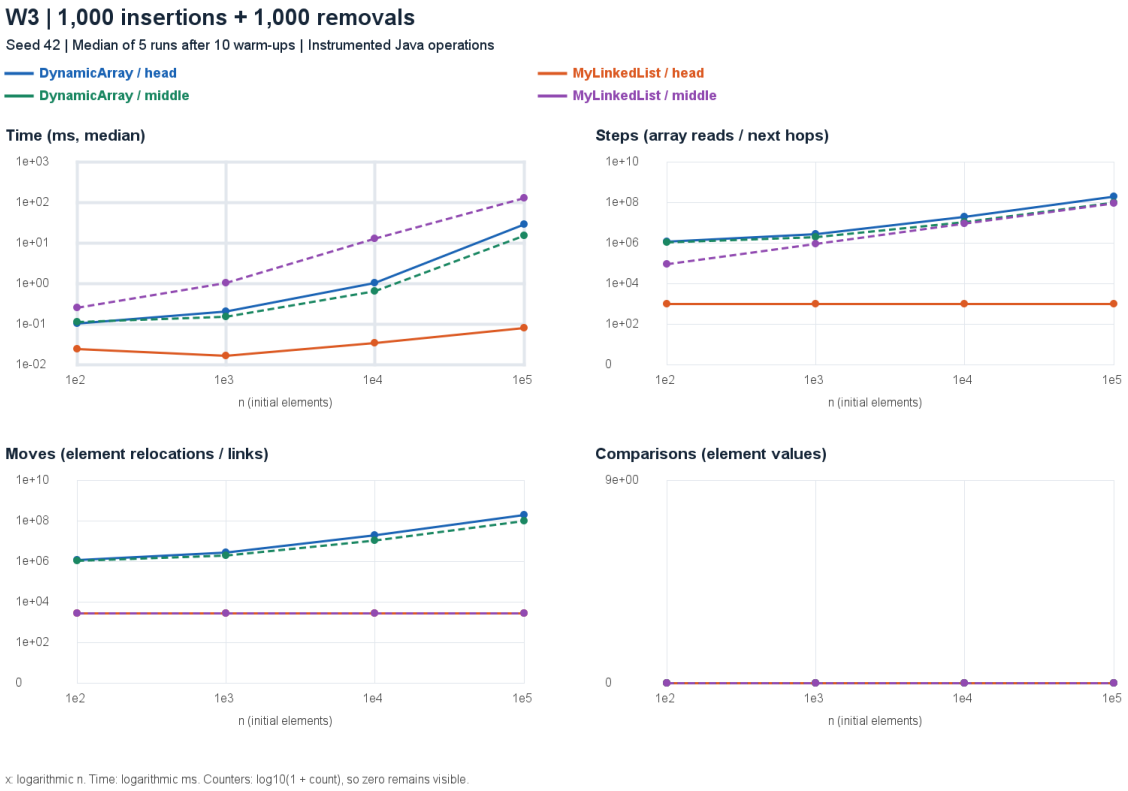
W1 at n=100,000: array 0.0372 ms, list 674.2536 ms; 10,000 array reads versus 504,930,938 next hops. Fixed query count makes array work independent of n. Short timings remain sensitive to JVM and scheduling effects.



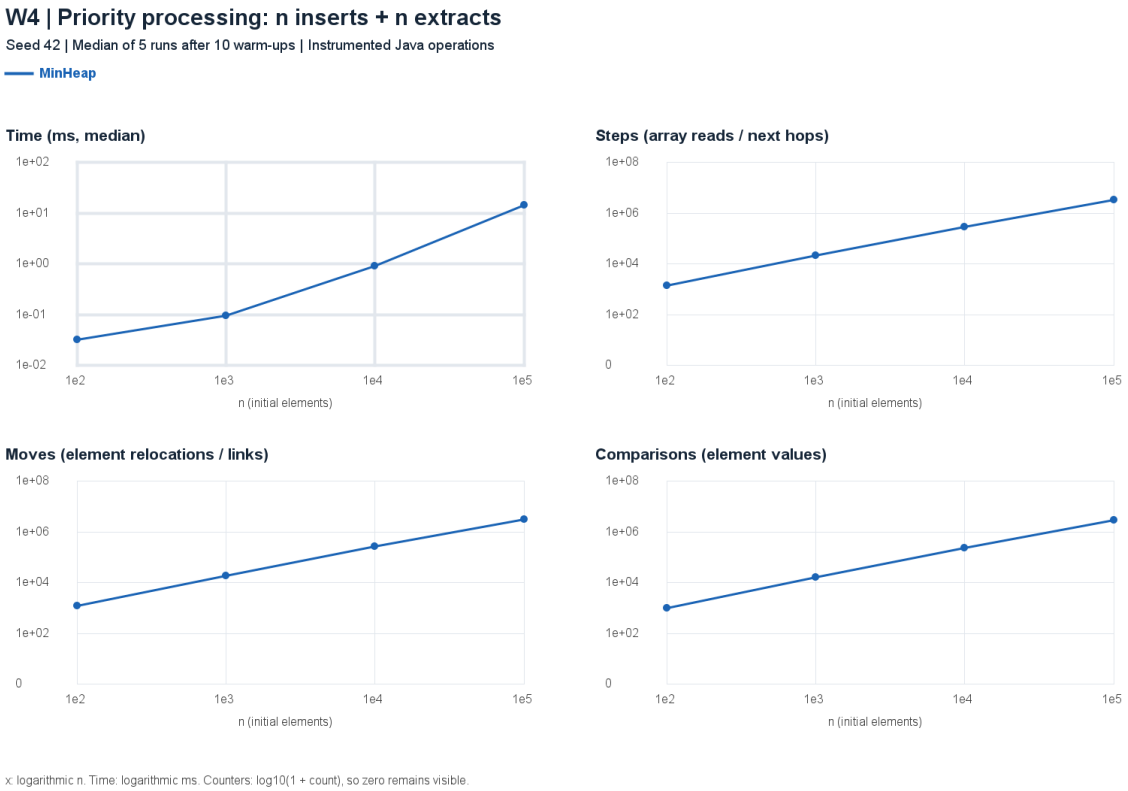
W2 at n=100,000: 52.8412 ms versus 132.0923 ms with the same 73,970,266 comparisons. List steps are exactly 500 fewer: successful queries return before following the matching node's next link. Log axes retain zero counters via log10(1+count); overlapping curves indicate equal counts.

Results: Mutation & Priorities

Akmaral Abilkhaiyr | SE-2518 | Design and Analysis of Algorithms



W3 at n=100,000: head 29.3270 ms (array) versus 0.0827 ms (list); middle 15.5675 versus 131.0665 ms. Array moves: 200,999,000 head / 100,999,000 middle. List: 3,000 moves for both; 1,000 / 99,999,000 steps. Small-n array cases also include growth copies.



W4 at n=100,000: 14.5899 ms and 3,059,125 comparisons; output is checked in every run. Only MinHeap is required for W4. Total time is expected Θ(n log n) for random data and O(n log n) worst-case; all-equal values can take Θ(n). Timings include instrumentation and are not direct CPU/cache measurements.

Trade-offs & Linear Heap Construction

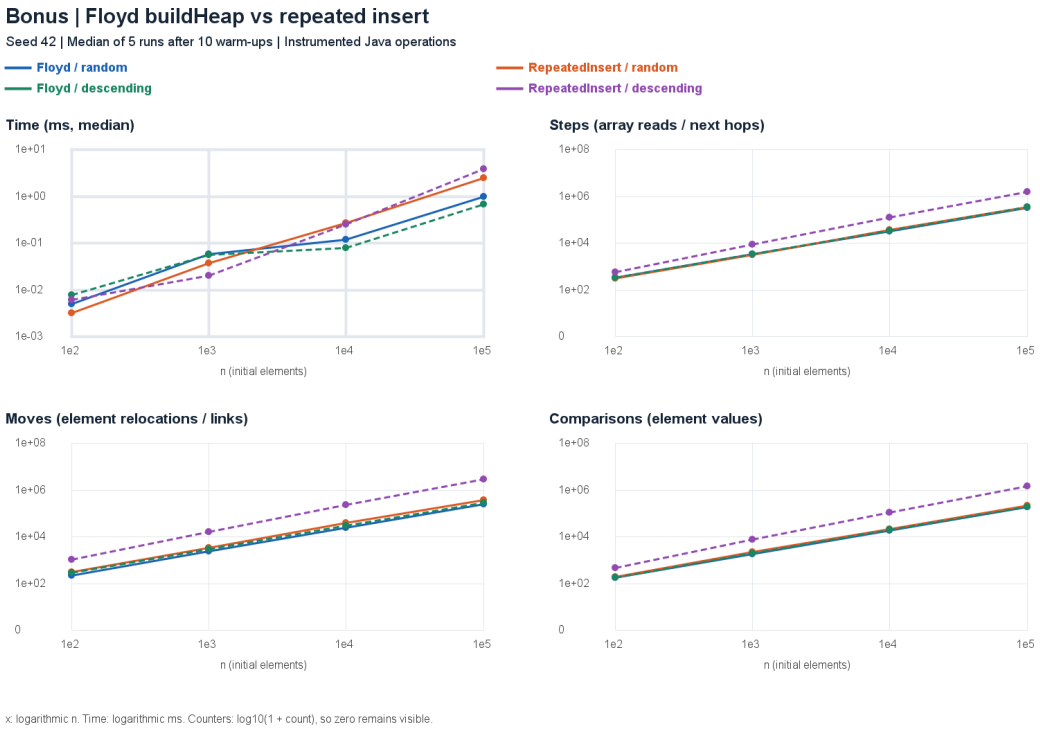
Akmaral Abilkhaiyr | SE-2518 | Design and Analysis of Algorithms

Discussion

DynamicArray reaches a random index directly, whereas this singly linked list follows every preceding next pointer. Consecutive primitive ints share cache lines, so array scans benefit from spatial locality and hardware prefetching. A list node can reside elsewhere in memory, and obtaining the next address depends on reading the current node. This pointer dependency can limit overlapping memory reads and explains why equal asymptotic bounds need not imply equal times. W2 provides the clearest example: equal comparison counts at n=100,000 accompany about 2.50 times longer list execution. Object headers, alignment and reference fields increase list memory traffic beyond the stored four-byte integer. Node allocations also create garbage-collector work, though this experiment does not separately measure collection time or cache misses. Head insertion and removal favor the list because a few pointer updates replace shifting almost the entire array. Middle insertion is not constant-time through an index-based list API because locating the predecessor is linear. The tail pointer makes append constant-time, but removing the tail still requires finding its predecessor. A heap is preferable for repeatedly scheduling the smallest-priority job because it maintains the minimum without fully sorting after each update. These explanations are consistent with the measurements, but timings also include counter updates, JIT effects and operating-system noise, so they do not prove a specific hardware cause.

Bonus B: why Floyd is linear

At most O(n/2^(h+1)) nodes have height h, so bottom-up sift work is O(n Σ h/2^h) = O(n). Reading/copying n inputs gives Ω(n), hence Θ(n). Repeated inserts cost Θ(n log n) on descending distinct keys, but expected Θ(n) on random-order data; the worst bound is not the cost of every input.



At n=100,000 descending keys: Floyd 199,978 comparisons / 0.6782 ms; repeated insert 1,468,946 / 3.9162 ms. Random keys: 188,424 / 1.0095 ms versus 227,662 / 2.5188 ms. Full extraction validates both results after timing. Bonus A/JOL was not measured; the memory discussion is qualitative.

Reproduce: mvn clean verify; java -jar target/daa-assignment2-1.0.jar. See README.md, results/raw-runs.csv and results/environment.txt. GitHub repository: akmaralabilhayir-lab/DAA_Assignment2_Akmaral_Abilkhaiyr_SE2518, branch main, tag v1.0. Reruns change time values; this report describes the committed run.